Chapter 02: Time Integration Methods for ODEs

From Forward Euler to Runge–Kutta Schemes

Pakorn Wongwaitayakornkul, PhD

PHY653: Computational Electromagnetics and Plasma Physics

Outline

Solving ODEs Numerically

Many physical systems are described by ordinary differential equations (ODEs):

$$\frac{d\mathbf{y}}{dt} = f(\mathbf{y}, t)$$

The fundamental problem: Given an initial condition $\mathbf{y}(t_0) = \mathbf{y}_0$, find $\mathbf{y}(t)$ for $t > t_0$.

Key idea: Discretize time into steps $t_n = t_0 + n\Delta t$ and approximate the solution at each step:

$$\mathbf{y}_0 \longrightarrow \mathbf{y}_1 \longrightarrow \mathbf{y}_2 \longrightarrow \cdots \longrightarrow \mathbf{y}_N$$

Different **time integration methods** give different rules for computing $\mathbf{y}_{n+1}$ from $\mathbf{y}_n$.

Time Integration Methods: Overview

Goal: Advance the solution from $\mathbf{y}_n$ to $\mathbf{y}_{n+1}$ using the derivative $f(\mathbf{y}, t)$.

Key properties to consider:

- **Accuracy** — Order of the truncation error ($\mathcal{O}(\Delta t^p)$)
- **Stability** — Does the numerical solution remain bounded?
- **Computational cost** — Number of function evaluations per step
- **Conservation** — Does the scheme preserve energy/momentum?

Methods we will study:

1. Forward Euler (explicit, 1st order)
2. Backward Euler (implicit, 1st order)
3. Leapfrog (implicit/symplectic, 2nd order)
4. Runge–Kutta 3rd and 4th order (explicit, higher order)

Explicit vs. Implicit Methods

Explicit Methods:
The next value y_{n+1} is computed *directly* from known quantities:

$$y_{n+1} = y_n + \Delta t \cdot f(t_n, y_n)$$

- Easy to implement
- Computationally cheap per step
- May require small Δt for stability

Implicit Methods:
The next value y_{n+1} appears on *both sides*:

$$y_{n+1} = y_n + \Delta t \cdot f(t_{n+1}, y_{n+1})$$

- Requires solving an equation for y_{n+1}
- Better stability properties
- Can use larger Δt
- Handles stiff systems well

Trade-off: Explicit methods are simpler but may be unstable; implicit methods are stable but more expensive per step.

Our Example: The Lorentz Equation

The motion of a charged particle in electromagnetic fields:

$$\frac{d\mathbf{v}}{dt} = \frac{q}{m}(\mathbf{E} + \mathbf{v} \times \mathbf{B})$$

Discretized form:

$$\Delta\mathbf{v} = \frac{q}{m}(\mathbf{E} + \mathbf{v} \times \mathbf{B}) \cdot \Delta t, \qquad \Delta\mathbf{x} = \mathbf{v} \cdot \Delta t$$

State vector representation: Define $\mathbf{y} = (x, y, v_x, v_y)$, then:

$$\frac{d\mathbf{y}}{dt} = \begin{pmatrix} v_x \\ v_y \\ (q/m)(E_x - v_y B_z) \\ (q/m)(E_y + v_x B_z) \end{pmatrix}$$

This is our ODE system $\dot{\mathbf{y}} = f(\mathbf{y}, t)$ that we will integrate numerically.

Cross-Field Configuration: $\mathbf{E} \times \mathbf{B}$

Setup: Constant crossed fields $\mathbf{E} = E_y\,\hat{\mathbf{j}}$ and $\mathbf{B} = B_z\,\hat{\mathbf{k}}$.

Analytical solution (cycloid motion):

$$x(t) = R\,(\omega t - \sin\omega t)$$
$$y(t) = R\,(1 - \cos\omega t)$$

where:

- $\omega = \dfrac{qB}{m}$ — cyclotron frequency
- $R = \dfrac{mE}{qB^2}$ — cycloid radius

This exact solution lets us **benchmark** all our numerical methods against the known answer.

Defining the ODE System in Python

The state derivative function `state_dot`:

```
def state_dot(state: np.ndarray) -> np.ndarray:
    return np.array([
        state[2],           # dx/dt = vx
        state[3],           # dy/dt = vy
        (q/m) * (E - state[3] * B),  # dvx/dt
        (q/m) * (state[2] * B)       # dvy/dt
    ])
```

- `state` $= (x,\ y,\ v_x,\ v_y)$ — the 4-component state vector
- Returns $d\mathbf{y}/dt$ evaluated at the current state
- Used by all explicit integration methods

Forward Euler Method

The simplest explicit scheme (1st-order accurate):

$$\boxed{y_{n+1} = y_n + \Delta t \cdot f(t_n,\ y_n)}$$

Geometric interpretation: Follow the tangent line at (t_n, y_n) for a step of size Δt.

Properties:

- **Order:** 1 — local truncation error is $\mathcal{O}(\Delta t^2)$, global error $\mathcal{O}(\Delta t)$
- **Function evaluations:** 1 per step
- **Stability:** Conditionally stable — can blow up for oscillatory systems
- For the Lorentz equation: tends to **gain energy** over time (spirals outward)

Forward Euler: Implementation

```
def Euler_step(f, t, y, h):
    return y + h * f(t, y)
```

Applied to the Lorentz equation:

```
# Forward Euler update
for idx, t in enumerate(time):
    state += state_dot(state) * dt
    forward[:, idx] = state[0:2]
```

- One function evaluation per step
- Simple but accumulates error quickly
- Energy grows without bound for the cyclotron motion

Backward Euler Method

An implicit 1st-order scheme:

$$\boxed{y_{n+1} = y_n + \Delta t \cdot f(t_{n+1},\ y_{n+1})}$$

y_{n+1} appears on both sides $\Rightarrow$ must solve an implicit equation.

Practical approach: Use a predictor step (forward Euler) then correct:

$$\tilde{y}_{n+1} = y_n + \Delta t \cdot f(t_n,\ y_n) \qquad \text{(predict)}$$
$$y_{n+1} = y_n + \Delta t \cdot f(t_{n+1},\ \tilde{y}_{n+1}) \qquad \text{(correct)}$$

Properties:

- **Order:** 1 — same order as forward Euler
- **Stability:** Unconditionally stable (A-stable)
- For the Lorentz equation: tends to **lose energy** over time (spirals inward)

Backward Euler: Implementation

```
# Backward Euler update (predictor-corrector)
for idx, t in enumerate(time):
    state2 += state_dot(
        state2 + state_dot(state2) * dt
    ) * dt
    backward[:, idx] = state2[0:2]
```

Average of Forward and Backward Euler (Heun's method):

```
def Heun_step(f, t, y, h):
    k1 = f(t, y)
    k2 = f(t + h, y + h * k1)
    return y + 0.5 * h * (k1 + k2)
```

- Averaging forward and backward gives 2nd-order accuracy
- Better energy conservation than either alone

Forward vs. Backward Euler: Energy Behavior

Forward Euler:

- Energy **increases** each step
- Trajectory spirals **outward**
- Unstable for long simulations

Backward Euler:

- Energy **decreases** each step
- Trajectory spirals **inward**
- Overdamped behavior

Average (Heun):

- Errors partially cancel
- Much better energy conservation
- 2nd-order accuracy

Key lesson: For oscillatory systems like the Lorentz equation, 1st-order methods have systematic energy drift. We need higher-order or symplectic methods.

Leapfrog Method

A 2nd-order symplectic scheme:

For the ODE $\frac{dy}{dt} = f(y, t)$, the leapfrog discretization is:

$$\boxed{\frac{y_{\text{new}} - y_{\text{old}}}{\Delta t} = f\left(\frac{y_{\text{new}} + y_{\text{old}}}{2},\ t\right)}$$

where $y_{\text{new}} = y(t + \Delta t/2)$ and $y_{\text{old}} = y(t - \Delta t/2)$.

This is **implicit** in y_{new}, but for the Lorentz equation it can be solved analytically!

Properties:

- **Order:** 2 — global error $\mathcal{O}(\Delta t^2)$
- **Symplectic:** Exactly conserves a discrete energy (no long-term drift)
- **Time-reversible:** Swapping $\Delta t \to -\Delta t$ reverses the trajectory

Leapfrog for the Lorentz Equation

Define $\boldsymbol{\Omega} = q\mathbf{B}/m$ and $\boldsymbol{\Sigma} = q\mathbf{E}/m$. The Lorentz force becomes:

$$\frac{d\mathbf{v}}{dt} = \boldsymbol{\Sigma} + \mathbf{v} \times \boldsymbol{\Omega}$$

Applying leapfrog gives the implicit equation:

$$\mathbf{v}_{\text{new}} + \mathbf{A} \times \mathbf{v}_{\text{new}} = \mathbf{C}$$

where $\mathbf{A} = \boldsymbol{\Omega}\,\Delta t/2$ and $\mathbf{C} = \mathbf{v}_{\text{old}} + \Delta t\,(\boldsymbol{\Sigma} + \mathbf{v}_{\text{old}} \times \boldsymbol{\Omega}/2)$.

Closed-form solution:

$$\boxed{\mathbf{v}_{\text{new}} = \frac{\mathbf{C} + \mathbf{A}\,(\mathbf{A} \cdot \mathbf{C}) - \mathbf{A} \times \mathbf{C}}{1 + A^2}}$$

Position update: $\mathbf{x}_{\text{new}} = \mathbf{x}_{\text{old}} + \mathbf{v}_{\text{new}}\,\Delta t$.

Leapfrog: Cross-Field Update Equations

For $\mathbf{E} = E_y\,\hat{\mathbf{j}}$, $\mathbf{B} = B_z\,\hat{\mathbf{k}}$, the velocity update simplifies to:

$$v_x' = \frac{v_x + v_y\,\Omega\,\Delta t}{1 + (\Omega\,\Delta t/2)^2}$$

$$v_y' = \frac{v_y - v_x\,\Omega\,\Delta t + \Sigma\,\Delta t}{1 + (\Omega\,\Delta t/2)^2}$$

where $\Omega = qB/m$ and $\Sigma = qE/m$.

Advantages for plasma simulations:

- No energy drift over long times
- Exact for uniform circular motion (gyration)
- The standard "Boris pusher" in PIC codes is based on this idea

Leapfrog: Implementation

```
def leapfrog_method(dt, num_steps, E, B):
    x_lf = np.zeros(num_steps)
    y_lf = np.zeros(num_steps)
    vx, vy = 0.0, 0.0

    sigma = (q/m) * E
    omega = (q/m) * B
    denom = 1.0 + (omega * dt / 2)**2

    for i in range(1, num_steps):
        # Leapfrog velocity update
        vx_new = (vx + vy*omega*dt) / denom
        vy_new = (vy - vx*omega*dt
                  + sigma*dt) / denom
        # Position update
        x_lf[i] = x_lf[i-1] + vx_new * dt
        y_lf[i] = y_lf[i-1] + vy_new * dt
```

Runge–Kutta Methods: Idea

Idea: Evaluate the derivative at *multiple points* within each step, then take a weighted average for higher accuracy.

General form:

$$y_{n+1} = y_n + \Delta t \sum_{i=1}^{s} b_i \, k_i$$

where each k_i is a derivative evaluation at a strategically chosen point.

Hierarchy of Runge–Kutta methods:

- **RK1** = Forward Euler — 1 evaluation, 1st order
- **RK2** = Midpoint / Heun — 2 evaluations, 2nd order
- **RK3** — 3 evaluations, 3rd order
- **RK4** — 4 evaluations, 4th order (the "classic" method)

Runge–Kutta 4th Order (RK4)

The classic RK4 algorithm:

$$
\begin{aligned}
k_1 &= h\, f(t_n,\ y_n) \\
k_2 &= h\, f\left(t_n + \tfrac{h}{2},\ y_n + \tfrac{k_1}{2}\right) \\
k_3 &= h\, f\left(t_n + \tfrac{h}{2},\ y_n + \tfrac{k_2}{2}\right) \\
k_4 &= h\, f(t_n + h,\ y_n + k_3)
\end{aligned}
$$

$$
\boxed{y_{n+1} = y_n + \frac{k_1 + 2k_2 + 2k_3 + k_4}{6}}
$$

Properties:

- **Order:** 4 — global error $\mathcal{O}(\Delta t^4)$
- **Function evaluations:** 4 per step
- Very accurate, the workhorse of scientific computing

Runge–Kutta 3rd Order (RK3)

A 3rd-order variant:

$$
\begin{aligned}
k_1 &= f(t_n,\ y_n) \\
k_2 &= f\left(t_n + \tfrac{h}{2},\ y_n + \tfrac{h}{2}\,k_1\right) \\
k_3 &= f(t_n + h,\ y_n + h\,(2k_2 - k_1))
\end{aligned}
$$

$$
\boxed{y_{n+1} = y_n + \frac{h}{6}\,(k_1 + 4k_2 + k_3)}
$$

Properties:

- **Order:** 3 — global error $\mathcal{O}(\Delta t^3)$
- **Function evaluations:** 3 per step
- Good compromise between cost and accuracy

Runge–Kutta: Implementation (`methods.py`)

```
def RK3_step(f, t, y, h):
    k1 = f(t, y)
    k2 = f(t + 0.5*h, y + 0.5*h*k1)
    k3 = f(t + h, y + h*(2*k2 - k1))
    return y + h*(k1 + 4*k2 + k3) / 6

def RK4_step(f, t, y, h):
    k1 = f(t, y)
    k2 = f(t + 0.5*h, y + 0.5*h*k1)
    k3 = f(t + 0.5*h, y + 0.5*h*k2)
    k4 = f(t + h, y + h*k3)
    return y + h*(k1 + 2*k2 + 2*k3 + k4) / 6
```

- All methods in `methods.py` share the same interface: `step(f, t, y, h)`
- Modular design — easy to swap methods

Complete Method Library (`methods.py`)

```
def Euler_step(f, t, y, h):
    return y + h * f(t, y)

def Heun_step(f, t, y, h):
    k1 = f(t, y)
    k2 = f(t + h, y + h * k1)
    return y + 0.5 * h * (k1 + k2)

def RK2_step(f, t, y, h):
    k1 = f(t, y)
    k2 = f(t + 0.5*h, y + 0.5*h*k1)
    return y + h * k2
```

- `Euler_step` — Forward Euler (1st order)
- `Heun_step` — Trapezoidal rule (2nd order)
- `RK2_step` — Midpoint method (2nd order)

Simulation Setup

```
# Field parameters
E, B, q, m = 1, 1, 1, 1

omega = q * B / m
R = m * E / (q * B**2)

T_max = 7 * 2 * np.pi / omega
num_steps = 1_000
dt = T_max / num_steps
time = np.linspace(0, T_max, num_steps)
```

- Normalized units: $q = m = E = B = 1$
- Cyclotron frequency: $\omega = 1$, cycloid radius: $R = 1$
- Simulate 7 full gyration periods

Comparing All Methods: Trajectory

All methods applied to the same cross-field cycloid problem:

Analytical solution:

$$x(t) = R(\omega t - \sin \omega t)$$
$$y(t) = R(1 - \cos \omega t)$$

Methods compared:

- Forward Euler (blue)
- Backward Euler (red)
- Average/Heun (green)
- Analytic (black dashed)

Observations:

- Forward Euler spirals outward (energy gain)
- Backward Euler spirals inward (energy loss)
- Average Euler closely tracks the analytic solution
- Higher-order methods (RK4) are nearly indistinguishable from analytic

Error Analysis

How does the error scale with time step Δt?

Define the error as the difference between numerical and analytical positions at $t = T_{\text{max}}$:

$$\text{Error} = |\mathbf{x}_{\text{numerical}}(T_{\text{max}}) - \mathbf{x}_{\text{analytic}}(T_{\text{max}})|$$

Expected scaling:

Method	**Order**	**Error $\sim$**
Forward/Backward Euler	1	$\mathcal{O}(\Delta t)$
Heun / Leapfrog	2	$\mathcal{O}(\Delta t^2)$
RK3	3	$\mathcal{O}(\Delta t^3)$
RK4	4	$\mathcal{O}(\Delta t^4)$

On a log-log plot of error vs. Δt, each method appears as a line with slope equal to its order.

Convergence: Error vs. Time Step

Log-log plot of error vs. number of time steps:

- Euler: slope ≈ -1
- Heun/Leapfrog: slope ≈ -2
- RK3: slope ≈ -3
- RK4: slope ≈ -4

Implication: Doubling the number of steps reduces error by:

- $2\times$ for Euler
- $4\times$ for Leapfrog
- $8\times$ for RK3
- $16\times$ for RK4

Exercise:
Reproduce the error convergence plot by running each method with varying numbers of time steps ($N = 10, 20, 50, 100, \ldots$) and computing:

$$\text{Error}(N) = \|\mathbf{x}_N - \mathbf{x}_{\text{exact}}\|$$

Verify the expected slopes on a log-log plot.

Key Takeaways

What we covered:

1. **ODE framework:** Reformulating physics as $d\mathbf{y}/dt = f(\mathbf{y}, t)$
2. **Explicit vs. Implicit:** Trade-offs between simplicity and stability
3. **Forward Euler:** Simple but gains energy (1st order)
4. **Backward Euler:** Stable but loses energy (1st order)
5. **Leapfrog:** Symplectic, conserves energy, exact for gyration (2nd order)
6. **Runge–Kutta (RK3/RK4):** High-order accuracy, the workhorse of ODE solvers
7. **Cross-field example:** Cycloid motion as a benchmark problem
8. **Error convergence:** Higher-order methods converge faster with Δt

Next steps: Apply these methods to more complex electromagnetic field configurations and multi-particle systems!

Key Equations Summary

Forward Euler:

$$y_{n+1} = y_n + \Delta t \cdot f(t_n, y_n)$$

Backward Euler:

$$y_{n+1} = y_n + \Delta t \cdot f(t_{n+1}, y_{n+1})$$

Leapfrog:

$$\frac{y_{\text{new}} - y_{\text{old}}}{\Delta t} = f\left(\frac{y_{\text{new}} + y_{\text{old}}}{2},\ t\right)$$

RK4:

$$y_{n+1} = y_n + \frac{k_1 + 2k_2 + 2k_3 + k_4}{6}$$